Project: Refactoring a parking garage application

1- There is a total of 4 actors that can ask for a change in ParkingGarage:

- The team that does the pricing
- The team responsible for payment
- The team responsible for reporting
- The Developers responsible for writing, changing files...

2-

Principle	**Class**	**Line(s) of code**	**Smell**
Single Responsibility Principle	ParkingGarage	Lines for classes: fee, checkout, cancel, dailyReport	God class, change requested by different teams: pricing, payment, reporting teams and developers.
Open / Closed Principle	ParkingGarage	The switch(vehicle.getType()).	Since a new vehicle type forces the edit of fee()
Liskov Substitution Principle	CashPayment	The method refund() throws an exception: Unsupported-OperationException.	Needs to override an exception otherwise it cannot do it safely
Interface Segregation Principle	PaymentMethod	The line of void refund(double amount)	Fat interface, it forces refund on payment methods that cannot be refundable
Dependency Inversion Principle	ParkingGarage	The line of: new FileLogger("parking.log").	The business logic creates a variable with new

3- The existing files that must be edited with the current design is :

a : ParkingGarage.java (We should add the case of the vehicle BUS to the switch in fee()).

b : Main.java (In order to use this new refundable mobile_wallet payment method).

c : ParkingGarage.java and Main (since we now use a FileLogger in ParkingGarage, so it must be edited to use a database logger instead, and also Main would change to call the DatabaseLogger)

d : ParkingGarage.java (since the daily report is built inside ParkingGarage.java the editing should be there).

4- The program in this case crashes. CashPayment breaks the principle of Liskov Substitution Principle. The instanceof test is not an acceptable fix because it just hides that violation instead of removing it, since any other client that calls refund through PaymentMethod will crash as well.

5- It is impossible because checkout ends with a logger.log(receipt), however logger is created inside ParkingGarage using "new", since the test can't replace it, every checkout appends a line to the file.

8- In my design, if a developer tries to cancel a cash payment through ParkingService, the code will not compile, because cancel only works for RefundablePayment, and CashPayment implements only PaymentMethod, leading to the compile error.

For question 4, the same error used to crash the code at runtime with UnsupportedOperationException. But now the error is caught at compile time before the program runs.

9- The test double that we need is a Receipt log (Logged) that stores receipts, in order to pass it in ParkingService through its constructor.

10- The existing files I had to open are:

a: Main in order to create new vehicle type BUS.

b: Main in order to pass the mobile wallet payment to checkout.

c: Main in order to pass it to ParkingService to be logged.

d: Main in order to pass CsvFormatter to the ParkingService constructor.